

Indian Institute of Technology Kharagpur

AUTUMN Semester, 2026–2027

COMPUTER SCIENCE AND ENGINEERING

Computer Organization and Architecture Laboratory

Verilog Assignment 4

August 2026

Instructions: *This is a group assignment intended for practice. Each group must implement the complete Verilog design, write a suitable testbench, and demonstrate the working design and its behavioral simulation to a Teaching Assistant for evaluation. The complete circuit must be purely combinational.*

Question 1: Hamming (7,4) Encoder and Syndrome Generator

Design and simulate a combinational Hamming (7,4) system that:

1. accepts four data bits;
2. generates a 7-bit Hamming codeword using even parity;
3. allows a single-bit error to be introduced for simulation;
4. checks the received 7-bit word; and
5. generates a 3-bit syndrome identifying the position of a single erroneous bit.

1. Basic Idea of Hamming (7,4)

A Hamming (7,4) code converts

4 data bits

into

7 transmitted bits.

The additional three bits are parity bits used for detecting and locating a single-bit error. The seven transmitted positions are numbered

1, 2, 3, 4, 5, 6, 7.

The parity bits are placed at positions that are powers of two:

1, 2, 4.

The remaining positions

3, 5, 6, 7

contain the four information bits.
Let

$$\text{data}[3:0] = d_3d_2d_1d_0.$$

The Hamming word is organized as follows:

Hamming Position	Binary Position Number	Contents	Verilog Bit
1	001	p_1	codeword[0]
2	010	p_2	codeword[1]
3	011	d_0	codeword[2]
4	100	p_4	codeword[3]
5	101	d_1	codeword[4]
6	110	d_2	codeword[5]
7	111	d_3	codeword[6]

Therefore,

$$\text{codeword}[6:0] = \{d_3, d_2, d_1, p_4, d_0, p_2, p_1\}.$$

2. Why There Are Three Parity Groups

Each parity bit checks a particular group of Hamming positions.

The groups are determined from the binary representation of the position number.

- p_1 checks every position whose least significant binary position bit is 1.
- p_2 checks every position whose second binary position bit is 1.
- p_4 checks every position whose third binary position bit is 1.

Therefore:

Parity Check	Hamming Positions Included
p_1	1, 3, 5, 7
p_2	2, 3, 6, 7
p_4	4, 5, 6, 7

Notice, for example, that Hamming position 6 has binary position number

$$6 = (110)_2.$$

Therefore, position 6 belongs to the p_2 and p_4 parity groups, but not to the p_1 parity group. This property will later allow the syndrome to identify the location of an erroneous bit.

3. Hamming (7,4) Encoder

Create a module named

`hamming74_encoder`

with the following interface:

Signal	Width	Direction
data	4 bits	Input
codeword	7 bits	Output

Even parity must be used.

The parity equations are:

$$p_1 = d_0 \oplus d_1 \oplus d_3,$$

$$p_2 = d_0 \oplus d_2 \oplus d_3,$$

and

$$p_4 = d_1 \oplus d_2 \oplus d_3.$$

These equations ensure that every parity group contains an even number of logic 1s.

Worked Encoding Example

Consider

$$\text{data} = 1011.$$

Therefore,

$$d_3 = 1, \quad d_2 = 0, \quad d_1 = 1, \quad d_0 = 1.$$

The first parity bit is

$$p_1 = d_0 \oplus d_1 \oplus d_3 = 1 \oplus 1 \oplus 1 = 1.$$

The second parity bit is

$$p_2 = d_0 \oplus d_2 \oplus d_3 = 1 \oplus 0 \oplus 1 = 0.$$

The third parity bit is

$$p_4 = d_1 \oplus d_2 \oplus d_3 = 1 \oplus 0 \oplus 1 = 0.$$

The seven Hamming positions are therefore:

Position 7	Position 6	Position 5	Position 4	Position 3	Position 2	Position 1
d_3	d_2	d_1	p_4	d_0	p_2	p_1
1	0	1	0	1	0	1

Hence,

$$\boxed{\text{codeword} = 1010101}$$

We can verify the three parity groups:

$$p_1 \text{ group: positions } 1, 3, 5, 7 = 1, 1, 1, 1,$$

which contains four logic 1s.

$$p_2 \text{ group: positions } 2, 3, 6, 7 = 0, 1, 0, 1,$$

which contains two logic 1s.

p_4 group: positions 4, 5, 6, 7 = 0, 1, 0, 1,

which also contains two logic 1s.

Thus all three groups have even parity.

4. Syndrome Generator

Create a module named

`syndrome_generator`

with the following interface:

Signal	Width	Direction
received	7 bits	Input
syndrome	3 bits	Output
error_detected	1 bit	Output

The syndrome generator checks the same three parity groups used during encoding. Let the received bits at Hamming positions 1 through 7 be

$$r_1, r_2, \dots, r_7.$$

The three check bits are

$$s_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7,$$

$$s_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7,$$

and

$$s_4 = r_4 \oplus r_5 \oplus r_6 \oplus r_7.$$

The syndrome is formed as

$$\boxed{\text{syndrome} = \{s_4, s_2, s_1\}}$$

If the received word is correct, all three parity checks pass:

$$s_4 = s_2 = s_1 = 0.$$

Therefore,

$$\text{syndrome} = 000.$$

5. Why the Syndrome Gives the Error Position

Suppose exactly one transmitted bit changes.

The parity checks containing that bit will fail.

Because the membership of each Hamming position in the parity groups is equal to the binary representation of the position number, the three syndrome bits directly form the binary error position.

For example:

Error Position	Binary	s_4	s_2	s_1
1	001	0	0	1
2	010	0	1	0
3	011	0	1	1
4	100	1	0	0
5	101	1	0	1
6	110	1	1	0
7	111	1	1	1

Therefore:

syndrome	Meaning
000	No error
001	Error at Hamming position 1
010	Error at Hamming position 2
011	Error at Hamming position 3
100	Error at Hamming position 4
101	Error at Hamming position 5
110	Error at Hamming position 6
111	Error at Hamming position 7

The output `error_detected` must be:

$$\text{error_detected} = 0$$

when the syndrome is 000, and

$$\text{error_detected} = 1$$

when the syndrome is non-zero.

6. Error Injection Using an Error Mask

In an actual communication system, a transmitted bit may change because of noise or another physical error.

During Verilog simulation, such an error must be deliberately created so that the error-detection circuit can be tested.

For this purpose, use a 7-bit signal

$$\text{error_mask}[6:0].$$

The error mask is **not part of the Hamming coding algorithm**. It is only used to imitate corruption of the transmitted codeword during simulation.

The received word is defined as

$$\boxed{\text{received_codeword} = \text{encoded_codeword} \oplus \text{error_mask}}$$

An XOR operation has the useful property

$$x \oplus 0 = x$$

and

$$x \oplus 1 = \bar{x}.$$

Therefore:

- a 0 in the error mask leaves the corresponding codeword bit unchanged;
- a 1 in the error mask flips the corresponding codeword bit.

For no error,

$$\text{error_mask} = 0000000.$$

For a single-bit error, exactly one mask bit must be 1.

The relation between the Hamming position and error mask is:

Hamming Position to Flip	Verilog Bit	error_mask
1	[0]	0000001
2	[1]	0000010
3	[2]	0000100
4	[3]	0001000
5	[4]	0010000
6	[5]	0100000
7	[6]	1000000

7. Complete Worked Error Example

Continue with the earlier data:

$$\text{data} = 1011.$$

The encoder generated

$$\text{encoded_codeword} = 1010101.$$

First consider the case with no error:

$$\text{error_mask} = 0000000.$$

Therefore,

$$\begin{aligned} \text{received_codeword} &= 1010101 \oplus 0000000 \\ &= 1010101. \end{aligned}$$

All parity checks pass, so

$$\text{syndrome} = 000$$

and

$$\text{error_detected} = 0.$$

Now suppose the bit at **Hamming position 6** is corrupted.

Position 6 corresponds to

$$\text{codeword}[5].$$

Therefore, use

$$\text{error_mask} = 0100000.$$

The received word becomes

$$\begin{aligned}\text{received_codeword} &= 1010101 \oplus 0100000 \\ &= 1110101.\end{aligned}$$

Now check the three parity groups.
For the s_1 check:

$$s_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7 = 1 \oplus 1 \oplus 1 \oplus 1 = 0.$$

Thus the p_1 group still passes.
For the s_2 check:

$$s_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7 = 0 \oplus 1 \oplus 1 \oplus 1 = 1.$$

Thus the p_2 group fails.
For the s_4 check:

$$s_4 = r_4 \oplus r_5 \oplus r_6 \oplus r_7 = 0 \oplus 1 \oplus 1 \oplus 1 = 1.$$

Thus the p_4 group also fails.
The syndrome is therefore

$$\text{syndrome} = \{s_4, s_2, s_1\} = 110.$$

Since

$$(110)_2 = 6,$$

the syndrome correctly identifies that the corrupted bit is at

$$\boxed{\text{Hamming position 6}}.$$

This can also be understood directly from the parity groups:

$$6 = (110)_2.$$

Position 6 belongs to the p_2 and p_4 groups, but not the p_1 group.
Therefore,

$$s_4 = 1, \quad s_2 = 1, \quad s_1 = 0,$$

giving

$$\text{syndrome} = 110.$$

8. Top-Level Hamming System

Create a top-level module named

`hamming74_system`

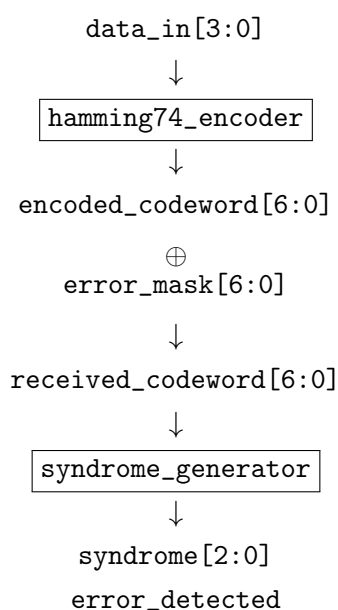
with the following interface:

Signal	Width	Direction
data_in	4 bits	Input
error_mask	7 bits	Input
encoded_codeword	7 bits	Output
received_codeword	7 bits	Output
syndrome	3 bits	Output
error_detected	1 bit	Output

The top-level module must instantiate:

- one `hamming74_encoder`;
- XOR logic for error injection; and
- one `syndrome_generator`.

The complete data flow is:



9. Logic-Driven Design Requirement

The encoder and syndrome generator must be implemented as **combinational logic networks**. Students may use:

- Verilog `xor`, `or`, and `not` gate primitives;
- continuous Boolean assignments using `assign`;
- wires for intermediate signals; and
- module instantiation.

The Hamming-code generation and syndrome generation must **not** be implemented using:

- `case` statements;
- large `if-else` mappings; or
- manually entered lookup tables.

The purpose of the assignment is to realize the Hamming parity equations directly using combinational logic.

No clocked or sequential logic is required.

10. Testbench Requirement

Write a testbench for the top-level `hamming74_system` module.

The testbench must perform the following tests.

1. Apply multiple 4-bit input values with

`error_mask = 0000000`

and verify

`syndrome = 000`

and

`error_detected = 0.`

2. Select at least one input data value and individually introduce an error at each of the seven Hamming positions using the following masks:

`0000001, 0000010, 0000100, 0001000,`

`0010000, 0100000, 1000000.`

3. Verify that the corresponding syndrome values are:

`001, 010, 011, 100,`

`101, 110, 111.`

4. Verify that `error_detected` becomes 1 whenever a single-bit error is introduced.

The behavioral simulation waveform must show at least:

`data_in,        encoded_codeword,        error_mask,`
`received_codeword,        syndrome,        error_detected.`

Note: For this assignment, consider only the cases of no error or exactly one erroneous bit. The interpretation of the syndrome as an error position is valid under this single-bit-error assumption.

11. Demonstration

Demonstration is mandatory.

Each group must demonstrate the working Verilog design and behavioral simulation to a Teaching Assistant.

During the demonstration, students may be asked to explain:

- why parity bits are placed at Hamming positions 1, 2, and 4;
- how the three parity groups are formed;

- how p_1 , p_2 , and p_4 are generated;
- why a correct codeword produces syndrome 000;
- why the syndrome identifies a single erroneous bit;
- how the error mask flips a selected codeword bit; and
- why the complete design is combinational.

12. Submission

Submit all Verilog design source files and the testbench in a single zipped folder named

`VerilogAssignment4_{Group_Number}.zip`.

The submission must contain:

- `hamming74_encoder`;
- `syndrome_generator`;
- `hamming74_system`; and
- the testbench for `hamming74_system`.

Each source file must contain the group number, names and roll numbers of all group members, assignment number, and date of submission.